目标：

1）明确jdk、jre、jvm三者之间的联系

2）如何读class字节码文件

3）class类的生命周期

4）class类加载机制

5）类加载器

6）双亲委派机制是否能够打破？如何打破？打破的场景

1 JDK、JRE、JVM

1.1 jdk

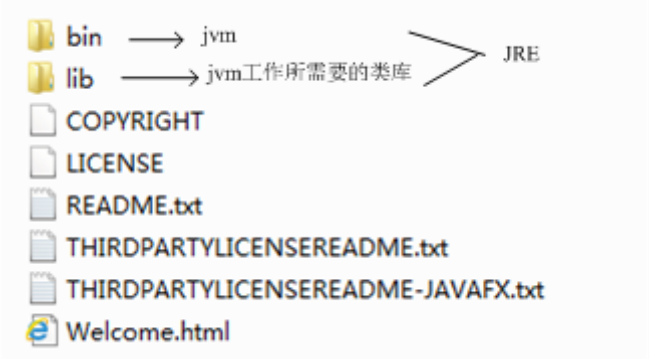
JDK(Java SE Development Kit)

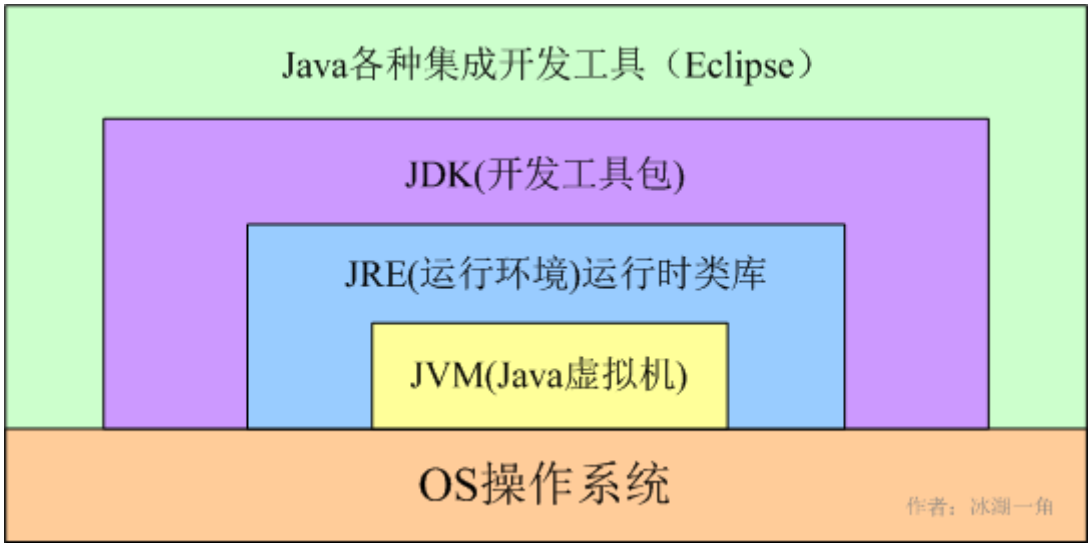
通过记事本去写java代码，写完之后，要运行，首先得通过javac这个命令进行编译

jdk比jre多了一个编译的功能

1.2 JRE

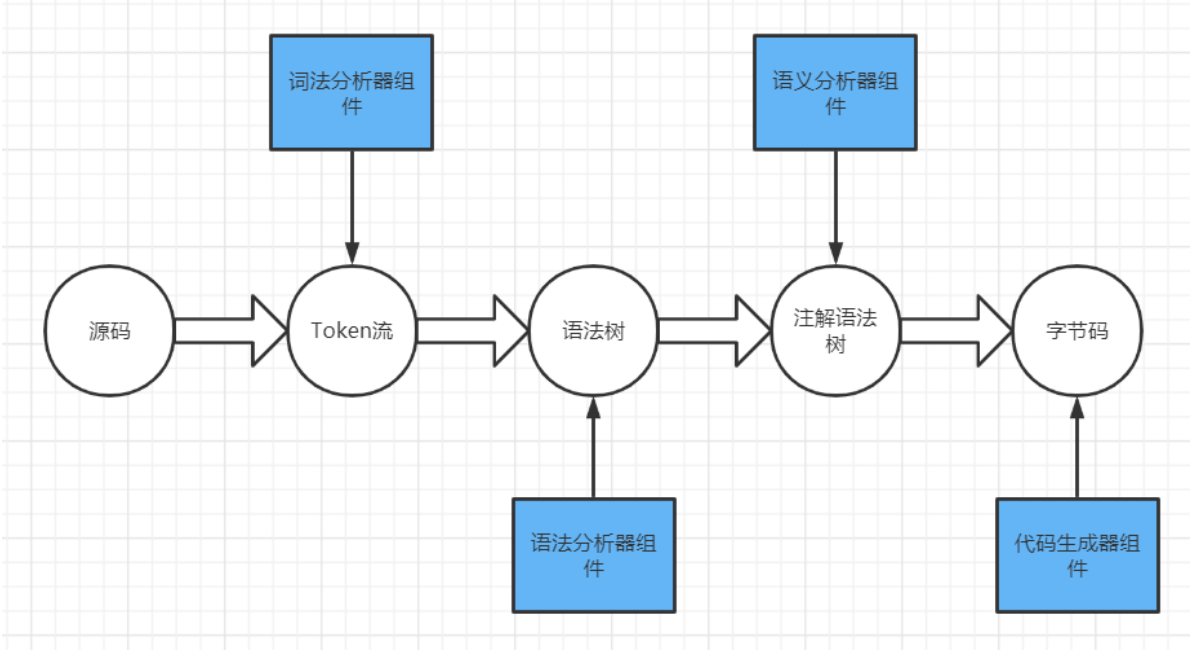
JRE(Java Runtime Environment)





2 java文件变class文件

javac xxx.java



词法分析器：就是分析代码里面有没有错别的关键字，哪些是名词，哪些是标点符号；

语法分析器：检查语法是否正确

语义分析器：foreach

代码生成器：转成字节码

不官是什么样的代码，只要能在JVM中执行，就一定要满足classfile结构

这就体现一次编译，到处运行（JVM帮我们实现了很多操作）start0 通知os去创建线程

```
ClassFile {
    u4              magic;    //魔法数，Class文件的标识。值是固定的，为0xCAFEBABE
    u2              minor_version;    //小版本
    u2              major_version;  //主版本号 34，52对应java 8 ,也就是我们jdk8生成的
class版本
    u2              constant_pool_count;    //38  常量池计算器
```

```
    cp_info         constant_pool[constant_pool_count-1];    //常量池
    u2              access_flags;  //访问标志   标志了类或者接口的访问信息，比如是类还是接口还是注解、枚举，是否是abstract,如果是类，是否被声明成final等等
    u2              this_class;  //类索引
    u2              super_class; //父类索引
    u2              interfaces_count;    //接口计数器   ，这也是之前大家在讨论动态代理的时候最多是65535个接口的时候讨论过的
    u2              interfaces[interfaces_count];
    u2              fields_count;    //字段个数
    field_info      fields[fields_count];  //字段集合
    u2              methods_count;    //方法计数器
    method_info     methods[methods_count];  //方法集合
    u2              attributes_count;  //附加属性计数器
    attribute_info attributes[attributes_count];     //附加属性集合
}
```

u4 u2： u表示的是无符号数 后面的数字表示占用了几个字节

_info：表示是一个表的结构，就是用来存储一堆数据的

magic：

1个字节换算成2进制占8位： 1111 1111

1个字节换算成16进制占2位： （15） （15）

magic （u4）：CAFE BABE （咖啡宝贝）

bean：咖啡豆 java：爪哇

minor_version（u2):00 00 --->0（小版本号）

major_version (u2)：00 ff ---->52 （主版本号）

java1.0版本：45

java1.1版本：45

从1.2--46 1.3--47 1.4---48 1.5--49 1.6--50 1.7--51 1.8--52

constant_pool_count:00 2A --->42(常量池大小) 这里的常量池跟字符串常量池不一样

42 表示当前常量池里面有41个常量

在jdk1.7之前，常量池里面的常量只有11种类型，在jdk1.7的时候，增加了3种，所以目前为止，常量池里面的常量只可能是14种

0A --->10(标志位)

00 09 ---> 9

00 22 ---->34

3 class类的生命周期

要运行一个class文件，肯定是需要通过JVM去执行的

3.1 加载

就是将磁盘中的class文件加载到内存里面去

3.1.1 类加载器

1）启动类加载器：它是JVM自己通过C++语言实现的，没有父类

2）扩展类加载器：

3）系统/应用类加载器：

4）自定义类加载器

3.1.2 加载机制

1. 全盘负责：所谓全盘负责，就是当一个类加载器负责加载某个Class时，该Class所依赖和引用其他Class也将由该类加载器负责载入，除非显示使用另外一个类加载器来载入。
2. 双亲委派：所谓的双亲委派，则是先让父类加载器试图加载该Class，只有在父类加载器无法加载该类时才尝试从自己的类路径中加载该类。通俗的讲，就是某个特定的类加载器在接到加载类的请求时，首先将加载任务委托给父加载器，依次递归，如果父加载器可以完成类加载任务，就成功返回；只有父加载器无法完成此加载任务时，才自己去加载。
3. 缓存机制。缓存机制将会保证所有加载过的Class都会被缓存，当程序中需要使用某个Class时，类加载器先从缓存区中搜寻该Class，只有当缓存区中不存在该Class对象时，系统才会读取该类对应的二进制数据，并将其转换成Class对象，存入缓冲区中。这就是为很么修改了Class后，必须重新启动JVM，程序所做的修改才会生效的原因。

3.1.3 双亲委派机制是否能够打破？如何打破？打破的场景

双亲委派的优势：

1）避免类的重复加载

2）防止核心API被随意篡改

它的优势又带来了一个问题

在一个Tomcat的webapp目录下放多个项目（多个war或者jar）

当两个项目引入了同一个组件的不同版本（也就是名字相同的class文件，但是内部代码不一样）

此时就不能满足需求了，只能打破双亲委派

3.2 验证

验证是不是咖啡宝贝，别的宝贝不执行，验证版本号，验证常量

3.3 准备

买房给宝贝住

3.4 解析

将常量池里面的符号引用转化成直接引用

3.5 初始化

给静态变量赋真正的值

3.6 使用

new Object()

3.7 卸载

Class被回收要满足以下三个条件：Girl.class

a)　　No Instance：该类所有的实例都已经被GC；不存在任何一个通过该类创建的对象

b)　　No ClassLoader：加载该类的ClassLoader实例已经被GC;

c)　　No Reference：该类的java.lang.Class对象没有被引用。(XXX.class, 静态变量/方法)

类对象(其实就是类)　类的对象（其实就是new出来的对象）

class类一旦被加载到jvm里面来之后，且通过了验证，那么就会产生一个引用指向该类

Class clazz = Girl.class